Acuity: Customer Intelligence & Retail Diagnostics Platform

Comprehensive Technical Architecture, Frontend, Backend Logic, REST APIs & Database Specification

Complete Technical Reference Guide for Academic Mini Project Final Report | Generated: October 28, 2026

Executive Purpose: This comprehensive reference document specifies the complete technical architecture, data pipelines, relational schemas, statistical formulations, and user interface systems implemented in **Acuity**. Acuity is an enterprise customer intelligence and retail revenue diagnostics platform. It ingests multi-megabyte retail transaction logs (such as the UCI Online Retail benchmark dataset), applies an automated 7-stage deterministic cleaning pipeline, computes multi-dimensional customer behavioral metrics (RFM Quintiles, Cohort Retention Matrices, Value-weighted Returns Leakage, Rolling Gaussian Anomaly Detection, Rule-Based Churn Risk Indices, and Inter-Purchase Gap Velocity), stores analytical entities in PostgreSQL with cascade integrity, and visualizes them on a modern, responsive React 18/TypeScript web dashboard with client-side 5-page executive PDF generation and full data export capabilities.

1. Complete Technology Stack & Specifications

Layer	Technology / Framework	Specific Version / Library	Role & Architecture Responsibilities
Frontend UI	React 18 + TypeScript	`React 18.3.1, TS 5.6`	Single Page Application (SPA), component architecture, strict type definitions.
Build Tool	Vite	`Vite 6.0`	Instant Hot Module Replacement (HMR), optimized production tree-shaken bundling.
Styling System	Tailwind CSS v4 + Tokens	`Tailwind 4.0, PostCSS`	Dark mode enterprise design system, CSS variables, glassmorphic UI cards, tabular numbers.
Visualizations	Apache ECharts 6	`echarts-for-react 3.0`	GPU-accelerated vector charting, dataZoom sliders, interactive multi-axis graphs.
Client State	Zustand	`zustand 5.0`	Lightweight global reactive stores for dataset status, active filters, drawer toggles.
Async Fetching	TanStack React Query	`@tanstack/react-query v5`	Automatic caching, background refetching, deduping API calls, loading states.
Backend Core	Python FastAPI	`FastAPI 0.143, Uvicorn 0.54`	Asynchronous ASGI REST API, automatic OpenAPI / Swagger documentation, request guards.
Analytics Core	Pandas + NumPy	`Pandas 2.3, NumPy 2.4`	High-speed vectorized array transformations, percentile ranking, time-series aggregations.
Statistical TS	Statsmodels (Optional)	`statsmodels 0.15`	STL (Seasonal and Trend decomposition using Loess) decomposition for weekly seasonality.
Database ORM	PostgreSQL + SQLAlchemy	`SQLAlchemy 2.1, Psycopg2`	Relational persistence, atomic transactions, foreign keys, cascade delete orphans.

PDF Generator	jsPDF + AutoTable	`jspdf 4.0, autotable 5.0`	Client-side vector generation of 5-page executive board report with charts and scorecards.
File IO & Export	OpenPyXL + CSV Stream	`openpyxl 3.1.5`	Parsing multi-MB Excel sheets, streaming CSV downloads, formatted clean Excel output.
Testing Suite	Pytest + SQLite In-Memory	`pytest 9.1, pytest-asyncio`	Unit and integration tests for data cleaning, analytics math, API routes, DB schemas.

2. System Architecture & Dual-State Processing Model

Acuity uses an asynchronous decoupled architecture designed for sub-second user responsiveness. Traditional web architectures often block the HTTP request while persisting large relational datasets, causing timeouts on 500k+ row spreadsheets. Acuity eliminates this bottleneck using a dual-state pipeline:

1. **Immediate In-Memory Analytics (Sub-Second Dashboard Readiness):** When a file is uploaded, the FastAPI backend immediately validates and cleans the data using vectorized Pandas routines. All 8 analytics engines execute in an isolated background thread managed by ThreadPoolExecutor(max_workers=1). The thread writes results into the thread-safe in-memory State singleton. As soon as in-memory state is ready, the frontend polling loop switches status to 'ready' and instantly renders all dashboards.
2. **Asynchronous Database Persistence:** Concurrently, a secondary background daemon persists all datasets, KPI records, monthly trends, RFM scores, cohort matrices, and customer risk entities into PostgreSQL inside an atomic database transaction. If the database is offline or temporarily unavailable, the system transparently operates on in-memory cache without crashing.
3. **Cold-Start Hydration:** When the backend service restarts, the API checks PostgreSQL for the most recently active dataset and rehydrates the dashboard overview metrics, ensuring data persistence across server restarts.
4. **Generation Counter (Concurrency Safety):** Each upload increments an atomic generation integer (state.generation). If a user uploads a new dataset while a previous upload is computing, the older pipeline aborts immediately, preventing race conditions or state pollution.

3. Complete Codebase Directory & File Structure

The project is organized into a Python FastAPI backend service and a React 18/TypeScript frontend client. The following table details every critical file, configuration, and module:

File / Directory Path	Subsystem	Detailed Role, Contents & Architecture Purpose

`.env`	**Environment**	Root configuration containing database host, port, user credentials, and API environment variables.
`README.md`	**Documentation**	Comprehensive project overview, quick-start setup guide, technology stack table, and architecture diagrams.
`package.json`	**Project Root**	Monorepo root package configuration facilitating concurrent workspace scripting.
`backend/requirements.txt`	**Backend Core**	Defines production Python dependencies: FastAPI, Uvicorn, Pandas, NumPy, SQLAlchemy, Psycopg2, OpenPyXL, etc.
`backend/storage/`	**Backend Storage**	Automated filesystem cache where uploaded raw CSVs and processed Excel workbooks (.xlsx) are persisted.
`backend/scripts/init_db.py`	**Backend DB Init**	Standalone schema initialization script that compiles all 11 SQLAlchemy models into PostgreSQL.
`backend/tests/conftest.py`	**Testing Suite**	Shared Pytest fixtures, test database engine setup, and synthetic transaction generators.
`backend/tests/test_cleaning.py`	**Testing Suite**	Unit tests validating header aliasing, deduplication, CustomerID presence, and cancellation pairing.
`backend/tests/test_analytics.py`	**Testing Suite**	Mathematical tests verifying RFM percentiles, cohort retention matrices, rolling z-scores, and risk scores.
`backend/tests/test_api.py`	**Testing Suite**	FastAPI endpoint integration tests verifying JSON contracts, query filters, and CSV export headers.
`backend/tests/test_database.py`	**Testing Suite**	ORM cascade tests confirming that deleting a dataset automatically wipes child records with zero orphans.
`backend/app/main.py`	**Backend Core**	FastAPI application entry point; configures CORS middleware and mounts all routes at /api/v1.
`backend/app/config.py`	**Backend Core**	Central settings; builds PostgreSQL connection URL; defines RFM weights, risk weights, anomaly thresholds.
`backend/app/database.py`	**Database Layer**	SQLAlchemy engine, session factory, Base declarative class, and active connection diagnostic probe.
`backend/app/models.py`	**Database Layer**	11 Relational ORM schemas with cascading delete integrity and indexes across IDs, dates, and segments.
`backend/app/schemas.py`	**Backend Core**	Pydantic models for request validation (e.g. RiskWeightParams) and structured API response bodies.
`backend/app/cleaning.py`	**Analytics Engine**	7-stage deterministic cleaning pipeline: standardizes columns, strips charges, pairs returns, splits datasets.
`backend/app/routes.py`	**API Layer**	All 19+ REST API route handlers, pagination helpers, customer 360 view, and streaming CSV/Excel exports.
`backend/app/services.py`	**Orchestration**	Manages upload pipeline execution in background threads; tracks generation counters to prevent race conditions.
`backend/app/db_service.py`	**Database Layer**	Executes atomic PostgreSQL transactions, sanitizes NaN/Inf values, and performs dataset queries.
`backend/app/state.py`	**State Layer**	Singleton in-memory state caching active DataFrames, KPIs, and computation status for sub-second responses.
`backend/app/utils.py`	**Utilities**	Generic pagination utility (paginate) and recursive NaN/Inf to None sanitizer (sanitize).
`backend/app/analytics/kpis.py`	**Analytics Core**	Calculates 9 headline KPIs, monthly sales trends, daily revenue, and narrative portfolio insights.
`backend/app/analytics/rfm.py`	**Analytics Core**	Computes Recency, Frequency, Monetary metrics; assigns 1-5 quintiles; classifies into 6 lifecycle segments.

backend/app/analytics/cohorts.py	**Analytics Core**	Builds 13-month triangle acquisition matrix tracking customer counts and revenue retention curves.
backend/app/analytics/returns.py	**Analytics Core**	Calculates return leakage by SKU, Segment, and Month against portfolio benchmark averages.
backend/app/analytics/anomalies.py	**Analytics Core**	Detects daily sales shocks using 30-day rolling Gaussian z-scores (|z| > 2.0) and attributes affected buyers.
backend/app/analytics/risk.py	**Analytics Core**	Rule-based composite 0-100 churn risk scoring combining RFM decay, returns, and gap widening.
backend/app/analytics/journey.py	**Analytics Core**	Computes order frequency, lifetime revenue, average inter-purchase gaps, and linear gap slopes.
backend/app/analytics/products.py	**Analytics Core**	Computes SKU return rates, repeat attach rates, and 4-quadrant matrix classification (Stars, etc.).
frontend/package.json	**Frontend Config**	Declares dependencies: React 18, Vite, Tailwind CSS v4, Apache ECharts, Zustand, jsPDF, Lucide icons.
frontend/vite.config.ts	**Build System**	Vite config defining path aliases (@/ -> src/) and reverse-proxying /api to FastAPI at port 8000.
frontend/src/main.tsx	**Frontend Entry**	React application root mounting QueryClientProvider and Toaster into document DOM.
frontend/src/app/router.tsx	**Routing System**	React Router v6 browser router definitions mapping all 14 feature routes under AppShell.
frontend/src/app/nav.ts	**Navigation**	Sidebar navigation registry defining primary routes, secondary routes, icons, and page descriptions.
frontend/src/app/layout/AppShell.tsx	**Layout System**	Root responsive layout frame containing Sidebar, Topbar, main Outlet, and MobileTabBar.
frontend/src/components/kpi/	**UI Library**	KpiCard and KpiGrid components rendering headline metrics, sparklines, and delta chips.
frontend/src/components/charts/	**Visualization**	ECharts wrapper, MonthlySalesChart, SegmentDonut, LeakageWaterfall, and ZoomSlider controls.
frontend/src/components/tables/	**UI Library**	Reusable, accessible, paginated DataTable component with sortable headers and custom cell renderers.
frontend/src/features/upload/	**Feature Module**	DataSourcePage, UploadCard (Cleaned vs Raw dropzones), ValidationPanel, and RecentDatasets table.
frontend/src/features/overview/	**Feature Module**	Executive Overview dashboard displaying KPI grid, monthly trend panels, segment mix, and leakage.
frontend/src/features/compare/	**Feature Module**	Side-by-side benchmarking page comparing active upload against historical Q3 baseline periods.
frontend/src/features/journey/	**Feature Module**	Customer Journey analysis with purchase interval distributions, Lorenz curve, and buyer table.
frontend/src/features/segments/	**Feature Module**	RFM customer segmentation page with interactive 3D bubble scatter plot and segment rules card.
frontend/src/features/cohorts/	**Feature Module**	13-Month acquisition cohort retention triangle heatmap and multi-cohort decay curves.
frontend/src/features/returns/	**Feature Module**	Returns intelligence dashboard analyzing margin leakage across SKUs, segments, and months.
frontend/src/features/products/	**Feature Module**	Product contribution matrix classifying catalog items into Stars, Loyalty Builders, and Long Tail.
frontend/src/features/anomalies/	**Feature Module**	Demand anomalies view graphing daily revenue vs rolling z-score with affected account attribution.

`frontend/src/features/risk/`	**Feature Module**	Customer churn risk radar featuring 4 dynamic factor weight sliders and high-risk account watchlist.
`frontend/src/features/simulator/`	**Feature Module**	What-If marketing campaign simulator calculating step-by-step incremental revenue projections.
`frontend/src/features/methodology/`	**Feature Module**	Mathematical documentation page providing formulas, worked examples, and metric definitions.
`frontend/src/features/mllab/`	**Feature Module**	Machine Learning Lab showcasing planned predictive models (CLV, Churn classifier, Market Basket).
`frontend/src/stores/`	**State Stores**	Zustand global stores: dataset.ts (polling status), filters.ts (active segments), ui.ts (drawer toggles).
`frontend/src/hooks/`	**Custom Hooks**	React hooks: useDataset, useBackendData (TanStack Query wrappers for each analytics endpoint).
`frontend/src/lib/pdf-report.ts`	**Reporting Core**	Client-side jsPDF reporting engine compiling 5-page board presentation with vector charts and tables.

4. Seven-Stage Deterministic Data Cleaning Pipeline

Real-world enterprise transaction logs contain missing customer identifiers, cancellations, shipping charges, currency symbol artifacts, and duplicated synchronization packets. The cleaning engine (backend/app/cleaning.py) implements a 7-stage deterministic pipeline. Detection is based on column names rather than file extensions or user flags.

Stage	Operation Name	Code Logic & Transformations	Purpose & Impact on Accuracy
Stage 1	**Header Normalization & Alias Mapping**	Converts all column headers to lowercase, strips trailing spaces, drops duplicate columns, and maps synonyms (e.g. client_id, customer_id → CustomerID; price, unit_price → UnitPrice; sku, item_code → StockCode).	Guarantees schema compatibility across disparate ERP and POS exports without manual user renaming.
Stage 2	**Deduplication**	Executes df.drop_duplicates() across all columns simultaneously.	Removes multi-channel replication retries and double-counted checkout sync packets.
Stage 3	**Type Sanitation & Parsing**	Coerces Quantity, UnitPrice, and CustomerID to numeric formats using pd.to_numeric(errors='coerce'). Parses InvoiceDate into ISO datetime objects.	Eliminates corrupt alphanumeric numbers, negative prices, and bad date formatting.
Stage 4	**Customer ID & Essential Validation**	Purges rows where CustomerID.isna(). Drops rows missing essential transaction keys (InvoiceNo, StockCode, Quantity, UnitPrice, InvoiceDate). Fills missing descriptions and countries with 'Unknown'.	Anonymous guest checkouts skew customer RFM and retention tracking; purging them isolates verifiable customer accounts.
Stage 5	**Non-Product Catalog Filtering**	Identifies non-inventory transaction codes: POST (Postage), DOT (Dotcom postage), M (Manual), S (Sample), AMAZONFEE, BANK CHARGES, C2 (Carriage), D (Discount), CRUK (Charity).	Prevents shipping fees and administrative banking charges from masquerading as physical merchandise sales.
Stage 6	**Transaction Segregation & Cancellation Pairing**	Rows where InvoiceNo starts with 'C' or Quantity < 0 are categorized as RETURN/CANCELLATION. All other rows are categorized as SALE.	Permits precise distinction between gross sales cash inflow and returned merchandise cash leakage.
Stage 7	**Financial Metric Derivation & Dataset Split**	Computes TransactionValue = Quantity * UnitPrice, SalesValue, ReturnValue, and NetRevenue = SalesValue - ReturnValue. Splits master dataset into sales_df and returns_df, each with positive LineValue.	Supplies normalized, pre-split tabular streams for all downstream analytics modules.

5. Core Analytics Engines & Mathematical Formulations

Acuity implements transparent, deterministic statistical formulations without black-box machine learning. Every number displayed on the dashboard can be traced directly to an exact mathematical formula:

5.1 RFM Customer Segmentation Engine (rfm.py)

Step 1 — Metric Aggregation: For each customer *c* across sales transactions:
• *Recency (R)* = ref_date - max(InvoiceDate_c) (in elapsed days, where ref_date = max(InvoiceDate) + 1 day)
• *Frequency (F)* = nunique(InvoiceNo_c) (count of distinct invoices)
• *Monetary (M)* = sum(LineValue_c) (total gross sales spend)

Step 2 — Quintile Scoring (1 to 5): Customers are assigned scores 1 to 5 based on empirical percentile ranks:
• R_Score = ceil(rank_pct(Recency, ascending=False) * 5) (Lower recency gets higher score 5)
• F_Score = ceil(rank_pct(Frequency, ascending=True) * 5) (Higher frequency gets higher score 5)
• M_Score = ceil(rank_pct(Monetary, ascending=True) * 5) (Higher spend gets higher score 5)
• RFM_Score = (R_Score * 0.40) + (F_Score * 0.30) + (M_Score * 0.30)

Step 3 — Sequential Top-Down Classification: A customer is categorized into the first matching segment:
1. **Champion:** R >= 4 AND F >= 4 AND M >= 4 (High recency, frequent buyers, top spenders)
2. **Loyal:** R >= 3 AND F >= 4 (Regular buyers with steady repeat cadence)
3. **New:** R >= 4 AND F <= 2 (Recent first-time purchasers requiring onboarding)
4. **At-Risk:** R <= 2 AND F >= 3 (Previously frequent buyers who have lapsed)
5. **Lost:** R <= 2 AND F <= 2 (Historical low-frequency buyers who have gone silent)
6. **Promising:** Default fallback for all remaining buyer profiles

5.2 Cohort Retention Matrix Engine (cohorts.py)

Customers are assigned to an acquisition cohort based on their first purchase month: CohortMonth = min(InvoiceDate_c).to_period('M').
For each subsequent calendar month *m*, the elapsed index is: MonthIndex = (Year_m - Year_cohort)*12 + (Month_m - Month_cohort).
• **Customer Retention %:** RetentionPct(c, i) = ActiveCustomers(c, i) / CohortSize(c, 0) * 100
• **Revenue Retention %:** RevenuePct(c, i) = Revenue(c, i) / Revenue(c, 0) * 100
The resulting matrix forms a triangle heatmap spanning 13 months, highlighting acquisition quality and revenue decay curves.

5.3 Returns & Revenue Leakage Intelligence (returns.py)

Evaluates cash leakage across three distinct operational dimensions: by StockCode (SKU), by Customer Segment, and by Calendar Month.
• **Dimension Return Rate:** ReturnRate_k = ReturnsValue_k / (SalesValue_k + ReturnsValue_k)
• **Portfolio Average Benchmark:** PortfolioAvg = sum(All_Returns) / (sum(All_Sales) + sum(All_Returns))
• **Leakage Flagging Condition:** An entity is flagged for margin risk if ReturnRate_k > PortfolioAvg.
Also computes individual customer return propensity Series: cust_return_rate = Returns_c / (Sales_c + Returns_c) for downstream risk scoring.

5.4 Demand Anomaly Detection Engine (anomalies.py)

Detects statistical shocks, demand spikes, flash sales, or operational dropouts in daily sales series Sales(t):
Mode 1 — 30-Day Rolling Gaussian Z-Score (Default):
For each day *t* with a preceding sliding window *W = 30 days* (minimum history *min_days = 7*):
• Mean_W = (1/N) * sum(Sales(i))
• Variance_W = max((1/N) * sum(Sales(i)^2) - Mean_W^2, 0.0), Std_W = sqrt(Variance_W)
• z_score(t) = (Sales(t) - Mean_W) / Std_W
• **Flagged Anomaly:** Flagged if |z_score(t)| > 2.0. Classified as spike (z > 0) or drop (z < 0).
• **Customer Attribution:** The engine joins sales logs to attribute all AffectedCustomerIDs who transacted on the anomaly date.
Mode 2 — STL Decomposition (Optional, requires statsmodels):
Decomposes daily sales into Trend, Weekly Seasonality (period = 7 days), and Residuals: Sales(t) = Trend(t) + Seasonal(t) + Resid(t). Flags dates where |z(Resid)| > 2.0.

5.5 Rule-Based Customer Churn Risk Engine (risk.py)

Computes a composite churn risk score (0 to 100) combining four normalized components (0.0 to 1.0):
1. **RFM Decay (w1 = 0.40):** rfm_decay = clip(1 - (RFM_Score - 1)/4, 0, 1) (Lower RFM score yields higher decay risk)
2. **Return Propensity (w2 = 0.25):** return_rate = clip(cust_return_rate, 0, 1) (Personal refund frequency)
3. **Purchase Gap Trend (w3 = 0.20):** gap_trend = clip(GapTrend / AvgGap, 0, 1) (Relative widening of purchase intervals)
4. **Anomaly Exposure (w4 = 0.15):** anomaly = 1.0 if customer transacted on any flagged anomaly date, else 0.0
• **Composite Score:** RiskScore = round(100 * (0.40*rfm_decay + 0.25*return_rate + 0.20*gap_trend + 0.15*anomaly), 2)
• **Risk Bands: Low** [0–25], **Medium** [25–50], **High** [50–75], **Critical** [75–100]
• **Main Driver Attribution:** MainReason = argmax(weighted_components) (explains exact primary risk cause).

5.6 Customer Journey & Purchase Gap Analysis (journey.py)

Aggregates transactions into discrete invoices per customer sorted chronologically. Calculates consecutive gaps in days: gaps = diff(InvoiceDates) / 1 day.
• AvgGap = mean(gaps) (average cadence between purchases)
• GapTrend = slope(linear_regression(x = 0..k, y = gaps))
A positive slope indicates purchase intervals are widening (cooling cadence and early churn indicator), while a negative slope indicates accelerating purchase frequency.

5.7 Product Catalog & Matrix Classification (products.py)

For each SKU (StockCode): computes GrossSales, ReturnedValue, NetRevenue, BuyerCount, TotalQtySold, and ReturnRate.
• **Repeat Attach Rate:** Fraction of buyers who purchased the SKU across 2 or more distinct invoices: AttachRate = mean(OrdersPerCustomer > 1).
• **4-Quadrant Matrix Classification (using portfolio median net revenue and attach rate):**
- **Stars:** High Net Revenue, High Repeat Attach Rate (Core revenue drivers)
- **Loyalty Builders:** Lower Net Revenue, High Repeat Attach Rate (Habit-forming products)
- **One-hit Sellers:** High Net Revenue, Low Repeat Attach Rate (Promotional flash sellers)
- **Long Tail:** Lower Net Revenue, Low Repeat Attach Rate (Catalog review candidates)

6. Relational Database Schema & SQLAlchemy ORM Models

The persistence layer is implemented in PostgreSQL using SQLAlchemy 2.0 ORM (backend/app/models.py). The schema comprises 11 relational tables centered on the datasets parent entity. All 10 child tables enforce strict foreign key constraints with ondelete='CASCADE' and SQLAlchemy cascade='all, delete-orphan', guaranteeing that purging a dataset automatically wipes all associated KPI records, RFM scores, cohort rows, and risk profiles in an atomic database transaction.

Table Name	ORM Model Class	Primary Columns & Data Types	Indexes & Constraints
datasets	DatasetModel	id (Int PK), dataset_name (VarChar 255), file_type (VarChar 50), original_rows (Int), final_rows (Int), sales_rows (Int), return_rows (Int), gross_sales (Float), total_returns (Float), net_revenue (Float), uploaded_at (Timestamp), is_active (Bool)	PK: id Index: dataset_name, is_active. Parent table for all child analytics.
dashboard_kpis	KpiModel	id (Int PK), dataset_id (Int FK), active_customers (Int), repeat_rate (Float), churn_rate_indicative (Float), rev_retention (Float), net_revenue (Float), gross_sales (Float), total_returns (Float), total_customers (Int), return_rate (Float), created_at (Timestamp)	FK: dataset_id → datasets.id (CASCADE). Index: dataset_id.
monthly_trends	MonthlyTrendModel	id (Int PK), dataset_id (Int FK), month (VarChar 10, YYYY-MM), sales (Float), returns (Float), return_rate (Float)	FK: dataset_id → datasets.id (CASCADE). Index: dataset_id, month.
daily_sales	DailySalesModel	id (Int PK), dataset_id (Int FK), sale_date (Date), sales (Float), mean (Float), std (Float), z_score (Float), flagged (Bool), direction (VarChar 50), affected_customer_ids (Text)	FK: dataset_id → datasets.id (CASCADE). Index: dataset_id, sale_date.
customer_journey	CustomerJourneyModel	id (Int PK), dataset_id (Int FK), customer_id (BigInt), order_count (Int), first_purchase_date (Date), last_purchase_date (Date), total_revenue (Float), avg_gap (Float), gap_trend (Float)	FK: dataset_id → datasets.id (CASCADE). Index: dataset_id, customer_id.
rfm_segments	RfmSegmentModel	id (Int PK), dataset_id (Int FK), customer_id (BigInt), recency (Int), frequency (Int), monetary (Float), r_score (Int), f_score (Int), m_score (Int), rfm_score (Int), segment (VarChar 50)	FK: dataset_id → datasets.id (CASCADE). Index: dataset_id, customer_id, segment.
cohort_retention	CohortRetentionModel	id (Int PK), dataset_id (Int FK), cohort_month (VarChar 10), month_index (Int), cohort_size (Int), retention_pct (Float), retention_rev_pct (Float)	FK: dataset_id → datasets.id (CASCADE). Index: dataset_id, cohort_month.
returns_intelligence	ReturnsIntelligenceModel	id (Int PK), dataset_id (Int FK), key_name (VarChar 100), key_type (VarChar 20: SKU, Segment, Month), sales (Float), returns (Float), return_rate (Float), flagged (Bool)	FK: dataset_id → datasets.id (CASCADE). Index: dataset_id, key_name, key_type.
product_contribution	ProductContributionModel	id (Int PK), dataset_id (Int FK), stock_code (VarChar 50), description (Text), gross_sales (Float), returned_value (Float), net_revenue (Float), buyers (Int), return_rate (Float), repeat_attach_rate (Float), category (VarChar 50)	FK: dataset_id → datasets.id (CASCADE). Index: dataset_id, stock_code, category.
anomalies	AnomalyModel	id (Int PK), dataset_id (Int FK), anomaly_date (Date), sales (Float), mean (Float), std (Float), z_score (Float), flagged (Bool), direction (VarChar 50), affected_customer_ids (Text)	FK: dataset_id → datasets.id (CASCADE). Index: dataset_id, anomaly_date.

customer_risk	CustomerRiskModel	id (Int PK), dataset_id (Int FK), customer_id (BigInt), risk_score (Float), risk_band (VarChar 20), main_reason (Text), rfm_decay (Float), return_rate (Float), gap_trend (Float), anomaly (Float)	FK: dataset_id → datasets.id (CASCADE). Index: dataset_id, customer_id, risk_band.

7. Comprehensive REST API Reference (All Endpoints)

All API endpoints are mounted under the prefix /api/v1 in backend/app/main.py. Standard responses are JSON serialized with NaN and Infinity values sanitized to null.

HTTP	Endpoint Route	Query / Body Parameters	Response Format & Description
GET	`/health`	None	Returns system status, service name, and PostgreSQL connection probe (connected, pool size, error).
POST	`/datasets/upload`	file: UploadFile (multipart form)	Accepts CSV, XLSX, Parquet. Starts background pipeline, returns fileType and cleaningSummary counts.
GET	`/datasets/status`	None	Polling endpoint returning status ('idle', 'processing', 'ready', 'failed'), datasetId, and cleaning summary.
GET	`/datasets`	None	Lists all stored datasets from PostgreSQL with row counts, upload timestamps, and active status flag.
GET	`/datasets/{id}`	Path: dataset_id (Int or string)	Returns comprehensive aggregated snapshot (KPIs, monthly series, segment shares, risk samples) by ID.
DELETE	`/datasets/{id}`	Path: dataset_id (Int)	Cascading deletion of dataset record and all child tables from PostgreSQL; purges cached disk files.
POST	`/datasets/clear`	None	Purges all stored datasets from PostgreSQL, frees disk storage cache, and resets in-memory singleton.
GET	`/datasets/overview`	None	Returns headline KPIs, monthly sales trends, segment mix distribution, and auto-generated textual insights.
GET	`/datasets/report-data`	None	High-performance server-side join (Risk + RFM + Journey) returning revenueAtRisk, watchlist, and segment shares for PDF generator.
GET	`/datasets/journey`	page, page_size, sort_by, order	Paginated customer journeys with order count, first/last purchase dates, revenue, avg gap, and gap trend slope.
GET	`/datasets/segments`	page, page_size, segment	Paginated RFM scores (R, F, M, RFM_Score) and segment distribution breakdown counts.
GET	`/datasets/cohorts`	None	13-month triangle cohort retention matrix containing both customer count retention and revenue retention.
GET	`/datasets/returns`	keyType: SKU|Segment|Month, flaggedOnly: Bool	Returns margin leakage diagnostics, portfolio benchmark average, and items flagged above the portfolio mean.

GET	`/datasets/products`	page, page_size, search, category, sort_by, order	Paginated catalog performance: GrossSales, ReturnedValue, NetRevenue, Buyers, ReturnRate, Category (Stars, etc.).
GET	`/datasets/products/{stock_code}`	Path: stock_code (string)	Detailed product view including monthly sales trajectory and order count history for a specific SKU.
GET	`/datasets/anomalies`	method: rolling|stl, flaggedOnly: Bool	Daily time series with rolling mean, std, z_score, flagged boolean, direction (spike/drop), and affected customers.
GET	`/datasets/risk`	page, page_size, band, rfm_decay, return_rate, gap_trend, anomaly	Rule-based churn risk scores (0–100) with dynamic query parameter factor weight overrides.
GET	`/datasets/customers/{customer_id}`	Path: customer_id (Int)	Customer 360 view: complete transaction history, RFM quintile scores, risk breakdown, and monthly purchase timeline.
GET	`/datasets/export/{table}.csv`	Path: table (journey, rfm, cohorts, returns, products, anomalies, risk)	Streams raw analytical data tables as downloadable CSV files with attachment Content-Disposition header.
GET	`/datasets/cleaned-file`	None	Streams sanitized production Excel workbook (.xlsx) containing normalized columns, dates, and LineValue.

8. Frontend Architecture & Complete Page Specifications

The frontend is structured into 14 distinct feature routes orchestrated by React Router v6. The global application layout (AppShell.tsx) provides a collapsible desktop sidebar, topbar with active dataset switching, and a mobile bottom navigation bar. Every feature page is detailed below:

Page Route	Feature Name	Visual Components & Charts	Analytical Focus & User Interactions
`/data-source`	**Data Source & Upload**	UploadCard (Cleaned & Raw dropzones), ValidationPanel, RecentDatasets table, Demo dataset loader.	File ingestion gateway; parses CSV/Excel; provides pre-upload column validation and instant status feedback.
`/overview`	**Executive Overview**	KpiGrid (6 metric cards with sparklines), MonthlySalesPanel (Sales vs Returns line), SegmentMixPanel (Donut), LeakagePanel (Waterfall).	High-level executive dashboard showing net revenue, churn rate, repeat rate, cash leakage, and primary portfolio KPIs.
`/compare`	**Dataset Benchmarking**	Dataset selector dropdown, side-by-side KPI comparison cards, Dual-line Monthly Sales Trajectory chart, Segment Migration grouped bar.	Benchmarks active upload against historical baseline (e.g. Q3 baseline or prior month snapshot); detects cohort drift.
`/journey`	**Customer Journey**	Order Frequency histogram, Purchase Interval Gap histogram, Customer Value Tiers (Lorenz curve), sortable Customer Table.	Tracks customer longevity, inter-purchase gaps (widening vs accelerating), and revenue concentration across VIP tiers.
`/segments`	**RFM Segmentation**	3D Bubble Scatter Plot (Recency vs Frequency scaled by Spend), Segment Filter Chips, RFM Rules legend, Paginated RFM table.	Clusters customers into 6 lifecycle tiers (Champion, Loyal, Promising, New, At-Risk, Lost) based on quintile scores.

`/cohorts`	**Cohort Retention**	Triangle Heatmap Matrix (13 months), Toggle between Customer % and Revenue %, Retention Decay curves, Cohort Size bars.	Measures long-term customer and revenue retention from initial acquisition month; pinpoints month-over-month drop-off.
`/returns`	**Returns & Leakage**	Breakdown Tabs (By SKU, By Segment, By Month), Return Rate Bar with Portfolio Benchmark line, Share of Sales vs Returns bars.	Identifies high-return merchandise and buyer segments disproportionately eroding gross profit margins.
`/products`	**Product Contribution**	Top-10 Revenue Products bar, 4-Quadrant Scatter Matrix (Return Rate vs Repeat Attach Rate), Product Catalog DataTable.	Classifies catalog items into Stars, Loyalty Builders, One-hit Sellers, and Long Tail to optimize merchandising.
`/anomalies`	**Demand Anomalies**	Dual-axis Daily Sales & Z-score bar chart, KPI counters (Flagged days, Spikes, Drops), Affected Customer Account list.	Detects statistical shocks (|z| > 2.0); attributes revenue spikes to wholesale buyers and drops to stockouts.
`/risk`	**Customer Risk Radar**	Risk Decile Distribution histogram, Segment Average Risk bars, 4 Interactive Weight Sliders, High-Risk Watchlist table.	Scores churn probability (0–100) per customer; allows executive adjustment of RFM decay, returns, and gap weights.
`/simulator`	**What-If Simulator**	Segment Selector buttons, Campaign Reach slider, Conversion Rate slider, Order Multiplier slider, Step-by-Step Revenue calculation.	Simulates marketing campaign ROI and projected incremental revenue across selected customer cohorts before execution.
`/methodology`	**Methodology & Equations**	Accordion topics with sticky sidebar table of contents, mathematical formula badges, concrete calculation examples.	Provides complete algorithmic transparency for all platform metrics; proves absence of black-box heuristics.
`/ml-lab`	**Machine Learning Lab**	Model Cards (CLV Regression, Churn Classification, Market Basket Apriori), 'Coming Soon' preview overlays.	Demonstrates roadmap expansion for advanced predictive machine learning models in future platform releases.
`/pipeline`	**Pipeline Status Scaffold**	Automatic redirect handler to /overview.	Maintains URL backwards compatibility for dataset upload workflows.

9. Five-Page Board-Ready PDF Presentation Report

In addition to on-screen interactive dashboards, Acuity includes a client-side vector PDF reporting engine (frontend/src/lib/pdf-report.ts) that converts analytics into a formatted 5-page executive presentation report for C-suite leadership:

Page	Executive Title	Sections, Scorecards & Tables	Business & Strategic Purpose
Page 1	**Executive Scorecard & Cashflow Reconciliation**	Corporate Header, 6 Core KPI scorecards (Net Revenue, Active Accounts, Repeat Rate, Churn Rate, Rev Retention, Return Rate), Leakage Waterfall bar, Monthly Net Revenue trajectory chart.	Gives Chief Commercial Officers immediate visibility into net cash capture, gross margin erosion, and seasonal trajectory.
Page 2	**Customer Lifecycle & RFM Segmentation**	Complete 6-tier RFM segment breakdown table (Account count, % share, Total Spend, AOV, Return Rate), Segment Revenue Concentration analysis, Quintile Scoring rule reference.	Enables marketing leaders to allocate retention budgets according to true customer value and migration risk.

Page 3	**Capital Exposure & Churn Risk Radar**	Total Capital Exposed metric banner, Risk Decile Distribution chart, Top-15 High-Risk Key Account Watchlist with Customer ID, Segment, Risk Score, Spend, and Prescribed Action.	Armors customer success teams with proactive retention targets to prevent high-value account churn.
Page 4	**Product Catalog Margin Leakage & Stars**	Top 10 Gross Revenue SKUs table, High-Return Leakage Catalog Lines table (units > 30, return rate > benchmark), Supplier QA Mitigation guidelines.	Highlights specific inventory lines causing refund drain to inform supplier negotiations and quality control.
Page 5	**Demand Shock Timeline & 90-Day Strategic Roadmap**	Chronological demand anomaly timeline (flagged date, shock direction, volume, affected buyers), Phased 90-Day Implementation Action Plan (Days 1–30, 31–60, 61–90).	Translates retrospective diagnostics into actionable operational initiatives and milestone deliverables.

10. Testing Suite, Verification & Deployment Guide

Acuity includes a test suite covering the entire pipeline. SQLite in-memory databases are utilized during automated testing to verify ORM schemas and cascading deletes without external database dependencies:

- backend/tests/test_cleaning.py: Validates header normalization, deduplication, removal of invalid customer IDs, negative price handling, and return segregation.
- backend/tests/test_analytics.py: Verifies mathematical accuracy of RFM quintiles, cohort retention percentages, rolling Gaussian z-scores, and rule-based risk calculations.
- backend/tests/test_database.py: Confirms that all 11 ORM models compile cleanly against SQLAlchemy and validates foreign key cascade deletion.
- backend/tests/test_api.py: Tests FastAPI endpoints using Starlette TestClient to guarantee JSON serialization and proper HTTP status codes.

```
QUICK START EXECUTION INSTRUCTIONS:
1. Backend Startup:
cd backend
python -m venv venv && .\venv\Scripts\activate (or source venv/bin/activate on Linux/Mac)
pip install -r requirements.txt
python scripts/init_db.py # Compiles PostgreSQL tables
uvicorn app.main:app --reload --port 8000

2. Frontend Startup:
cd frontend
npm install
npm run dev # Starts Vite dev server at http://localhost:3000

3. Production Build Verification:
npm run build # Compiles production TypeScript bundle into frontend/dist
```

Summary & Mini Project Conclusion: Acuity successfully bridges the divide between static reporting spreadsheets and opaque AI models by offering a fast, transparent, deterministic analytics pipeline. From ingestion to multi-page executive PDF rendering and PostgreSQL persistence, all subsystems are rigorously tested, type-safe, and ready for production or academic presentation.